

Research Document

Database

Casey Stijlaart

Table of Contents

1. Preface	1
2. Document history	2
3. Research Questions	3
4. Database Location	4
4.1. Local- or Cloud-based Database	4
4.2. Database Structure	5
5. Data Structure	7
5.1. Data Structure Techniques	7
5.2. Key-Value Stores	7
5.3. Recommendation	7
6. Database Storage Format	9
6.1. Document-Based Storage Format	9
6.2. Traditional Database Storage	9
6.3. Recommendation	10
7. Database Options	11
7.1. Requirements	11
7.2. Options	11
7.3. Recommendation	13
8. Recommendations and Conclusion	14

1. Preface

In this research document, there are critical aspects explored to find the most suitable database. This document covers important topics such as the choice between local and cloud-based databases, database structure, data structure techniques, database storage format, and various database options. The research provides a detailed analysis of the advantages and disadvantages of each option and offers valuable recommendations for the project's database needs.

2. Document history

Table 1. Document History

Version	Date	Status	Author	Description	Remarks
0.1	01-10-2023	Draft	Casey Stijlaart	Added Sprint 1 research (database structure, database format and data structure)	The research had previously already been done, but not yet put into one document.
0.2	03-11-2023	Draft	Casey Stijlaart	Added Sprint 2 research (database options)	The research had previously already been done, but not yet put into one document.

3. Research Questions

To guide the exploration and analysis for finding the most suitable database and its structure, the following research questions are established:

Main Research Question

- What is the most suitable database for our project's needs?

Sub-questions

- Should the database be local or cloud-based?
- Which database storage format is ideal?
- What should be the database structure?
- What data structure techniques should be employed?

4. Database Location

A database can be stored in multiple manners. Each manner having their own advantages and disadvantages.

4.1. Local- or Cloud-based Database

A database can be stored in two manners, locally or in the cloud. Both of these options having their own advantages and disadvantages, making an option more suitable for one purpose than another.

4.1.1. Local Database

Local databases refer to databases that are hosted and managed on-premises, typically within an organization's physical infrastructure. They are stored and maintained locally on the organization's servers or dedicated hardware.

Advantages

1. **Data Control:** The organization has full control over their data. Custom security measures and access controls can be set to their specific needs.
2. **Performance:** As the data is stored locally, the data can be accessed faster with a lower latency.
3. **Compliance:** Since data is stored locally, the data is secured by the better compliance options as the data does not leave the premises of the organization.

Disadvantages

1. **Cost:** As everything is locally, there is a dependency on hardware and staff for management. This can be costly for smaller projects.
2. **Scalability:** When large sizes of data is stored, the storage will run low and therefor must be expanded upon. This requires a significant investment and updates in software to manage the drives.
3. **Security:** While local databases offer control, they also demand more responsibility for security. There must be invested in a robust security to protect against physical and cyber threats.

4.1.2. Cloud-based Database

Cloud-based databases are hosted and managed by third-party cloud service providers, such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform. Data is stored and processed on remote servers maintained by these providers.

Advantages

1. **Scalability:** Cloud databases allow for easier scalability. The amount of storage and computing can be changed on-demand, ensuring there will always be enough storage.
2. **Accessibility:** As they are stored in the cloud, data can be accessed from anywhere with the help of an internet connection. This allows for more flexibility and possible remote work and

global collaborations.

3. **Automatic Updates:** Cloud providers handle updates, patches, and maintenance, reducing the burden on in-house IT teams. This ensures databases are running with the latest security features.

Disadvantages

1. **Data Security:** While cloud providers ensures the good security, a third party is involved. Meaning that the data is entrusted to the third party. Therefore, it's crucial to carefully manage access control, encryption, and compliance to mitigate potential risks.
2. **Data Transfer Costs:** Transmitting large volumes of data to and from the cloud can incur additional costs, especially if the organization has limited bandwidth or significant data transfer needs.
3. **Vendor Lock-In:** Moving away from a cloud provider can be complex and costly, creating a form of vendor lock-in. It's essential to plan for data portability and exit strategies.

4.1.3. Recommendation Database Storage

As the goal of the project is to create a prototype, there are certain aspects that are not yet relevant regarding to the database. A good example of such an aspect is the security and scalability. For a prototype a local database is a solid start. It allows for full control of the data, no internet dependency and is easier to setup.

4.2. Database Structure

As for the database itself, it has one main requirement, flags. Data must be retrievable based upon, so called, flags. These flags allow for data sets to be accessible for only the groups that have access to given flag. Another important aspect to be taken into account is the uncertainty of what data fields will be stored within When it comes to the type of data structure, there are two options, relational and non-relational.

4.2.1. Relational Databases (RDBMS)

A relational database is structured, meaning the data is organized in tables. Many times, the data within these tables have relationships with one another or dependencies.

Advantages

1. **Structured Data:** RDBMS excels in handling structured data with well-defined schemas.
2. **Data Integrity:** ACID properties ensure strong data integrity and consistency, making RDBMS suitable for applications where data accuracy is critical.
3. **Historical Data:** RDBMS systems are well-suited for creating a historical record of changes of changes made to data over time.

Disadvantages

1. **Rigid Schema:** The fixed schema of RDBMS can be limiting if your data sets evolve over time or if you need to store unstructured or semi-structured data.
2. **Scalability:** Scaling RDBMS systems can be challenging, especially for high-velocity data streams, which may be a limitation for real-time data collection.

4.2.2. Non-Relational Databases (NoSQL)

A non-relational database is document-oriented, meaning all information gets stored in more of a laundry list order. You will have all of your data listed within a single construct or document.

Advantages

1. **Flexible Schema:** NoSQL databases allow for a dynamic or flexible schema, making them suitable for storing data sets with varying or evolving data fields.
2. **Scalability:** NoSQL databases are designed for horizontal scalability, making them well-suited for real-time applications with rapidly changing data.
3. **High Throughput:** NoSQL databases can handle high volumes of data and can be optimized for high throughput, crucial for real-time data capture.

Disadvantages

1. **Eventual Consistency:** The BASE model, followed by NoSQL databases, may provide eventual consistency, which may not be suitable if strong data consistency is essential.
2. **Learning Curve:** Depending on the chosen NoSQL database type (e.g., document-oriented, key-value, graph), there may be a learning curve for adapting to the specific data model.

4.2.3. Recommendations and Conclusion Database Structure

Keeping further development for the database and data structure in mind, the recommendation is to make use of a non-relational as this allows for more flexibility. To clarify this, as the data sets may have fields added or removed dynamically, and considering the need for flexibility, scalability, a NoSQL database is more usable. An example of such is MongoDB, as its easy to adapt changes in data structure, capture real-time data, and scale your database as needed. MongoDB is a good recommendation for a NoSQL database that allows for encrypted data on the network and on disk

5. Data Structure

There are many types of data structures that can be used for a database. Each type is used for a different purpose. Examples of such are tables, indexes, trees and heaps. When picking a data structure it is of great importance to keep in mind in what type of database it is stored.

As discussed in [\[Recommendations Database Structure\]](#), the recommendation is to make use of an local non-relational database, NoSQL. This local database will be stored on a physical drive. Based upon this, it is a given that there is made use of flexible tables. Tables, though seemingly straightforward, offer a spectrum of techniques to enhance their utility.

Techniques of such are Key-Value Stores and indexes. These two techniques can enhance performance and overall clarity of the database.

5.1. Data Structure Techniques

5.2. Key-Value Stores

Key-value stores are a simple yet powerful data structure. They store data as pairs of keys and corresponding values. Key-value stores are highly efficient for data retrieval based on a unique identifier (the key).

Purpose

Key-value stores are particularly advantageous when rapid access to individual data components is essential, allowing for efficient data retrieval based on unique identifiers (the keys).

5.2.1. Indexes

An index in a non-relational database is a data structure that improves the speed of data retrieval operations on a table. It's like an additional table containing a subset of the data in the main table, organized for quick lookup based on one or more columns.

Purpose

Indexes are used to speed up the retrieval of rows from a table. Without an index, the database would need to perform a full table scan to find specific rows, which can be slow for large datasets. Indexes allow the database engine to locate the desired rows more efficiently.

Types of Indexes There are various types of indexes, including B-tree indexes (commonly used for most data types), hash indexes (for exact matches), and bitmap indexes (useful for columns with a limited set of possible values).

5.3. Recommendation

For the prototype, the recommendation is to make use of tables with the Key-Value Stores technique to efficiently store and retrieve data. If feasible, incorporating indexes, especially on relevant attributes such as "SensorID," is highly recommended. Indexes facilitate faster data processing,

ensuring that data can be received and retrieved in a continuous, uninterrupted stream.

6. Database Storage Format

When it comes to database storage, there are two primary approaches to consider: Document-Based Storage Format and Traditional Database Storage (NoSQL). These distinct methods offer their own set of advantages and disadvantages, and the choice between them is pivotal in shaping how data is managed and accessed in your application. To find the most suitable option, the two storage formats are laid out, compared and matched with the requirements. Based on the requirements and the gained knowledge, a recommendation is made.

6.1. Document-Based Storage Format

Document-based storage is a database storage approach that emphasizes flexibility and adaptability in data storage. In this model, data is stored in document format, typically using formats such as JSON or XML. This approach offers several advantages and disadvantages that need to be carefully considered.

6.1.1. Advantages

- **Flexibility:** Document-based storage formats are inherently flexible, making them ideal for handling semi-structured or unstructured data. This flexibility enables the addition or modification of fields without requiring extensive schema changes.
- **Schema Evolution:** The document-based approach allows for schema evolution, accommodating changing data requirements as the application evolves over time.
- **Rapid Development:** Document-based databases are often favored during the initial development stages, enabling rapid prototyping and iterations without rigid schema constraints.
- **Hierarchical Data:** It's well-suited for hierarchical or nested data structures, which are common in certain applications.

6.1.2. Disadvantages

- **Limited Query Capabilities:** Document-based databases may offer less powerful query capabilities compared to traditional databases. Complex queries and aggregations may require additional application-level processing.
- **Data Consistency:** Maintaining data consistency can be challenging in a schema-less environment, requiring careful application-level handling.
- **Performance:** Document-based databases may not be as optimized for complex querying and reporting tasks compared to traditional databases.

6.2. Traditional Database Storage

Traditional database storage, often categorized as NoSQL, encompasses a variety of database management systems designed to handle structured and semi-structured data. This approach provides a structured schema and offers several advantages and disadvantages.

6.2.1. Advantages

- **Structured Data:** Traditional databases enforce a structured schema, ensuring data consistency and integrity.
- **Complex Queries:** They excel at handling complex queries, reporting, and analytics, often supporting SQL for querying.
- **ACID Transactions:** Traditional databases are ACID-compliant, guaranteeing data consistency and reliability even in the face of failures.
- **Security and Compliance:** They offer robust security features, including access control and audit trails, which are crucial for sensitive data and compliance requirements.

6.2.2. Disadvantages

- **Schema Rigidity:** The structured schema can be less flexible and may require schema changes when data structures evolve.
- **Development Overhead:** Schema changes can lead to development overhead and potentially slow down the development process.
- **Scalability Challenges:** Some traditional databases may face challenges when it comes to horizontal scalability, especially in distributed environments.

6.3. Recommendation

The choice between document-based storage format and traditional database storage (NoSQL) hinges on the specific requirements of your application. In our case, where we prioritize flexibility, schema evolution, and real-time data processing, a document-based storage format is a strong candidate. However, the final decision should consider other factors such as performance, scalability, and the complexity of queries and reporting.

Therefore, based on our defined requirements and after a comprehensive evaluation of both storage options, we recommend adopting a document-based storage format for our database. This choice aligns with our need for flexibility, adaptability, and the capability to handle real-time data effectively.

7. Database Options

In our search for a database solution that aligns with our specific requirements, we have identified several document-based databases that not only meet these criteria but also offer the flexibility of hybrid and cloud-based deployments, as this could become of importance when the project grows. Below, we discuss the concluded requirements and explore the various options, considering their advantages and disadvantages in the context of our needs.

7.1. Requirements

Based on the research that has taken place, covering various aspects of the database, a set of requirements has been set up. The requirements are as following : - A NoSql database. - Document-based storage. - Built-in security and encryption. - Flexibility in scheme. - High performance. - Scalability. - Stored on an external drive. - Usage of various database techniques.

It is also important to keep in mind that the used database should be able to be integrated on the chosen hardware, raspberry pi 4. Therefor all the mentioned options are compatible.

7.2. Options

7.2.1. MongoDB

Advantages

- **NoSQL Database:** MongoDB is a leading NoSQL database, providing flexibility in schema and document-based storage.
- **Security and Encryption:** It offers robust security features, including authentication, role-based access control, and encryption at rest.
- **Flexibility:** MongoDB's schema-less design allows for agile schema changes and adaptation.
- **High Performance:** It is optimized for high-performance read and write operations.
- **Scalability:** MongoDB can be scaled horizontally to handle larger workloads and real-time data.
- **External Storage:** Data can be stored on an external drive, supporting your storage requirements.
- **Hybrid and Cloud:** MongoDB Atlas, the managed cloud service, offers seamless hybrid and cloud deployments.
- **Indexing and Views:** MongoDB supports indexing for efficient querying and flexible views using aggregation.

Disadvantages

- **Complex Queries:** Complex queries may require additional application-level processing.
- **Data Consistency:** Maintaining strong data consistency may need careful handling.
- **Operational Complexity:** Managing and configuring MongoDB can be complex in large-scale deployments.

7.2.2. Couchbase

Advantages

- **NoSQL Database:** Couchbase offers a NoSQL approach with flexible JSON document storage.
- **Security and Encryption:** It provides robust security features, including authentication and data encryption.
- **Flexibility:** Schema flexibility allows for accommodating changing data requirements.
- **High Performance:** Couchbase is known for its high-performance capabilities, including sub-document operations.
- **Scalability:** It supports horizontal scaling and real-time data scenarios.
- **External Storage:** Couchbase supports data storage on external drives.
- **Hybrid and Cloud:** Couchbase offers hybrid and cloud deployments with Couchbase Cloud.
- **Indexing and N1QL:** N1QL, a SQL-like query language, enables indexing and complex querying.

Disadvantages

- **Complex Views:** Creating and managing views may require advanced JavaScript skills.
- **Data Distribution:** Efficient data distribution and rebalancing can be challenging.

7.2.3. Amazon DynamoDB

Advantages

- **NoSQL Database:** DynamoDB is a managed NoSQL database with built-in scalability.
- **Security and Encryption:** It offers strong security features and encryption options.
- **Flexibility:** DynamoDB allows for flexible schemas and schema evolution.
- **High Performance:** It is designed for high-performance and low-latency access.
- **Scalability:** DynamoDB scales seamlessly to handle large-scale data and real-time requirements.
- **External Storage:** While it's primarily a cloud service, it supports external storage integration.
- **Hybrid and Cloud:** DynamoDB can be part of hybrid architectures and offers cloud-native features.
- **Indexing:** DynamoDB provides automatic indexing for efficient querying.

Disadvantages

- **Vendor Lock-In:** It's tightly integrated with AWS, which may lead to vendor lock-in.
- **Cost Considerations:** Costs can increase with large-scale usage and complex queries.

7.2.4. CouchDB

Advantages

- **NoSQL Database:** CouchDB is a NoSQL database with document-based storage.
- **Security and Encryption:** It offers basic security features and supports data encryption.
- **Flexibility:** CouchDB's schema flexibility aligns with changing data structures.
- **High Performance:** It performs well for simple queries and smaller-scale deployments.
- **Scalability:** It can handle moderate scalability requirements.
- **External Storage:** CouchDB can store data on external drives.
- **Hybrid and Cloud:** CouchDB can be deployed on various platforms, including cloud services.
- **Views:** CouchDB's views provide flexible querying capabilities.

Disadvantages

- **Limited Complex Queries:** Complex queries may require extensive application-level processing.
- **Data Distribution:** Efficient data distribution and replication can be challenging.
- **Security:** While it offers basic security, it may not be as robust as other options.

7.3. Recommendation

After careful consideration of the various database options and their suitability for integration into a C++ application running on a Raspberry Pi 4, we recommend MongoDB as the preferred choice. MongoDB aligns well with our project's requirements and offers several advantages that make it a strong candidate for our use case.

While MongoDB does have some disadvantages, such as the potential complexity of managing it in large-scale deployments and the need for additional application-level processing for complex queries, these limitations can be mitigated with careful planning and tuning, especially for a Raspberry Pi 4-based application.

In summary, MongoDB is a well-rounded database solution that aligns with our project's requirements, offers strong security features, and provides the necessary tools for integration into a C++ application on a Raspberry Pi 4. We believe that MongoDB is the most suitable option to meet our database needs and enable the successful development of our application on this platform.

8. Recommendations and Conclusion

In conclusion, the research has provided valuable insights into the selection of the most suitable database for the project. All of the set research questions were answered after the thorough research.

When reviewing the research questions and what their conclusion/answer is, the following can be concluded.

- For database location, a local database is recommended due to its data control and ease of setup.
- The database storage format is document-based due to its flexibility, schema evolution support, and suitability for real-time data.
- For the database structure, employing a non-relational database like MongoDB is recommended for its flexibility, scalability, and real-time data processing capabilities.
- The data structure should incorporate a key-value store with tables to enable flexible data retrieval based on flags.

After evaluating various database options, MongoDB is the preferred choice, offering alignment with our project's requirements, strong security features, high performance, scalability, and integration capabilities with the Raspberry Pi 4.